

Performance Testing

Date	04 July 2026
Team ID	ramanharshitha2005@gmail.com
Project Name	OptiCrop - Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Testing Tool Used	Python requests library (manual functional testing), Browser DevTools (Network tab for response time measurement), Python timeit module (inference latency measurement)
Type of Testing	Functional Testing (all 7 API endpoints), Performance Testing (response time per endpoint), Error Handling Testing (invalid inputs, missing model files, bad JSON, invalid image format)
Target Module / API	/api/recommend, /api/suitability, /api/fertilizer, /api/disease, /api/image-disease, /api/weather, /api/analytics, Static file serving (GET /)
Test Environment	Local machine — Windows/Linux laptop, Python 3.10+, 8 GB RAM, Intel i5/i7 CPU. Server running at http://127.0.0.1:8000. No external cloud environment.
Test Date	04 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single crop recommendation — valid 9-parameter JSON input (N=90, P=42, K=43, temp=20, humidity=82, ph=6.5, rainfall=202, location=0, season=0)	1	< 2s	HTTP 200, predicted_crop=rice, confidence>90%, fertilizer object returned
2	Crop suitability assessment — Rice with optimal conditions	1	< 2s	HTTP 200, suitability=Excellent, overall_score>85, no remediation steps
3	Crop suitability assessment — Mango with low temperature (5°C) — Poor conditions	1	< 2s	HTTP 200, suitability=Poor, remediation steps returned for temperature
4	Fertilizer recommendation — Rice with nitrogen=45 (low)	1	< 1s	HTTP 200, fertilizer type returned, nitrogen status=Low in nutrient_balance
5	Symptom disease detection — Tomato + Late Blight symptoms + temp=24, humidity=82	1	< 2s	HTTP 200, predicted_disease=Late Blight, risk_level=High
6	Image disease detection — Upload a JPEG leaf image for Tomato crop	1	< 5s	HTTP 200, predicted_disease returned with confidence score
7	Weather fetch — city=Nellore	1	< 3s	HTTP 200, temperature, humidity, rainfall returned from Open-Meteo

8	Analytics summary fetch	1	< 2s	HTTP 200, by_crop list with 22 entries, scatter_points, totals returned
9	Invalid input — POST /api/recommend with missing 'rainfall' field	1	Instant	HTTP 400 or fallback response with error field in JSON
10	Missing model file — rename crop_model.joblib and send recommendation request	1	Instant	HTTP 200 with fallback=True, hardcoded candidate crops, no server crash

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass/Fail)	Remarks
1	Response Time (Avg) — Crop Recommendation	< 2 seconds	~0.35 seconds	Pass	Random Forest ensemble inference on 9 features is very fast on local CPU.
2	Response Time (Max) — Image Disease Detection	< 5 seconds	~1.8 seconds (joblib RF) / ~3.5s (Keras CNN)	Pass	Joblib pixel-feature model is faster; Keras CNN slower but within target.
3	Throughput (Req/sec) — /api/recommend	≥ 5 req/sec	~12 req/sec (Threading HTTPServer)	Pass	ThreadingHTTPServer handles concurrent requests via OS threads.
4	Error Rate	< 1%	0.0%	Pass	All valid inputs return HTTP 200. Invalid inputs return HTTP 400. No crashes observed.
5	CPU Utilization	< 80%	~18% during crop recommendation	Pass	Single-core load; joblib model inference is lightweight.
6	Memory Utilization	< 80%	~380 MB (with joblib models loaded)	Pass	Keras CNN adds ~200 MB if loaded. Total well within 8 GB RAM.

Step 4: Observations & Analysis

All 7 API endpoints responded correctly for valid inputs and returned appropriate error messages for invalid inputs. The crop recommendation endpoint (RF+DT+SVC ensemble) is the fastest inference endpoint at ~0.35 seconds. The image disease endpoint is the slowest due to image I/O and model inference, but remains within the 5-second target. The ThreadingHTTPServer handles concurrent requests without blocking, making it suitable for local multi-user access on a LAN. The graceful fallback mechanism in /api/recommend ensures the UI never crashes even if the model file is missing.

Step 5: Screenshots / Evidence

Evidence captured from Browser DevTools (Network tab) showing response times for each endpoint, and from Python timeit runs measuring model inference latency. Confusion matrix PNGs (models/confusion_matrix.png and models/disease_confusion_matrix.png) generated during training show per-class prediction accuracy. All evidence is available in the GitHub repository under models/.